> aula_07.10

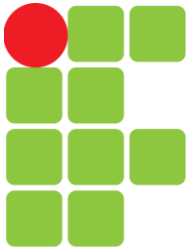
JavaScript

Parte 2 · decisões e funções

Desenvolvimento de Sistemas Web · 07/10

Prof. Thiago Araújo

IFPE Campus Palmares · ADS 3º período



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

> mapa_da_aula

Mapa da aula

01

`if`

Decisões: if e else

02

`for`

Repetições: laços

03

`fn`

Funções

04

`=>`

Arrow functions

05

`{ }`

Escopo

06

`2x`

Problemas em duplas

> pergunta?

Como o app do banco decide se o seu Pix passa?

01

Decisões: if e else

O código escolhe um caminho

`if` `for` `fn` `=>` `{ }` `2x`

> conceito

`if` executa um bloco **só se** a condição for verdadeira.

A condição vira `true` ou `false`. Lembra dos falsy?

```
const saldo = 50
const valor = 80

if (saldo >= valor) {
  console.log('Pix enviado!')
} else {
  console.log('Sem saldo')
}
```

> na_prática

Todo app decide com if

Banco · Pix

Saldo disponível

R$ 50,00

Valor do Pix

R$ 80,00

Enviar Pix

Saldo insuficiente

Todo app toma **decisões** com base em dados.

Saldo, idade, estoque, senha: tudo passa por um **`if`**.

```
if (saldo >= valor) pagar()
else avisarSemSaldo()
```

Várias saídas: else if

`if` — **Testa a primeira condição**
se for true, executa e para

`else if` — **Testa a próxima**
só se a anterior falhou

`else` — **O que sobrou**
nenhuma condição foi true

```
if (nota >= 9) {
  conceito = 'A'
} else if (nota >= 7) {
  conceito = 'B'
} else if (nota >= 6) {
  conceito = 'C'
} else {
  conceito = 'D'
}
```

Ternário: if que devolve valor

✗ Evite

```
let status
if (media >= 6) {
  status = 'aprovado'
} else {
  status = 'reprovado'
}
```

Seis linhas e um **`let`**.

✓ Prefira

```
const status = media >= 6
  ? 'aprovado'
  : 'reprovado'
```

Uma expressão, direto num **`const`**.

`switch` compara **um** valor com vários casos.

Esqueceu o **`break`**? Ele **continua** no próximo caso.

```
switch (opcao) {
  case '1':
    console.log('Saldo')
    break
  case '2':
    console.log('Extrato')
    break
  default:
    console.log('Inválida')
}
```




> checkpoint

O que o console mostra?

```
const idade = 17
idade >= 18 ? 'entra' : 'fica'
```

A `'entra'`

B `'fica'`

C `true`

D `undefined`

> resposta

O que o console mostra?

```
const idade = 17
idade >= 18 ? 'entra' : 'fica'
```

A `'entra'`

B `'fica'`

C `true`

D `undefined`

02

Repetições: laços

Fazer de novo sem copiar e colar

`if` `for` `fn` `=>` `{ }` `2x`

Laço repete um bloco **enquanto** a condição for verdadeira.

O computador não cansa: 10 ou 10 mil vezes.



Anatomia do for



Início

`let i = 0` roda uma vez

Condição

`i < 5`? Se false, sai

Corpo

executa o bloco { }

Passo

`i++` e volta ao 2

```
for (let i = 0; i < 5; i++) {
  console.log(`Volta ${i}`)   // Volta 0, Volta 1 ... Volta 4
}
```

Qual laço usar?

for

`for`

sabe quantas vezes vai repetir

while

`while`

repete enquanto a condição valer

of

`for...of`

visita cada item de uma lista

brk

`break`

sai do laço antes da hora



> na_prática

Somando o carrinho

loja.com/carrinho

Açaí 500 ml	**R$ 18,00**
Coxinha	**R$ 4,50**
Suco de caju	**R$ 6,00**
Total	**R$ 28,50**

`for...of` visita **cada item**, um por vez.

O `let total` acumula a soma.

```
const precos = [18, 4.5, 6]
let total = 0
for (const p of precos) {
  total += p
}
// total: 28.5
```

> erro_comum

O laço que nunca acaba

✗ Evite

```
let i = 0
while (i < 3) {
  console.log(i)
}
// trava a aba!
```

Esqueceu de mudar o `i`.

✓ Prefira

```
let i = 0
while (i < 3) {
  console.log(i)
  i++
}
```

Todo laço precisa **caminhar** até o fim.

> checkpoint

O que aparece no console?

```
for (let i = 1; i <= 3; i++) {
  console.log(i * 2)
}
```

A 1 2 3

B 2 4 6

C 2 4 6 8

D 0 2 4

> resposta

O que aparece no console?

```
for (let i = 1; i <= 3; i++) {
  console.log(i * 2)
}
```

A 1 2 3

B 2 4 6

C 2 4 6 8

D 0 2 4

03

Funções

Dar nome a um pedaço de lógica

`if` `for` `fn` `=>` `{ }` `2x`

Função é uma **receita**: recebe ingredientes e devolve um prato.

Escreva uma vez, use quantas vezes quiser.



Anatomia de uma função

`function` **Palavra-chave e nome**
um verbo que diz o que faz

`(a, b)` **Parâmetros**
os ingredientes de entrada

`return` **Retorno**
o prato: o valor de saída

`( )` **Chamada**
executa com valores reais

```
function media(a, b) {
  return (a + b) / 2
}

const m = media(7, 9)
console.log(m) // 8
```

Não se repita (DRY)

✗ Evite

```
const m1 = (7 + 9) / 2
const m2 = (5 + 8) / 2
const m3 = (6 + 10) / 2
// mudou a regra? 3 lugares
```

Copiar e colar multiplica o bug.

✓ Prefira

```
const m1 = media(7, 9)
const m2 = media(5, 8)
const m3 = media(6, 10)
// regra num lugar só
```

Mudou a regra? Muda **uma** função.



> na_prática

Parâmetro com valor padrão

Mercadinho do Seu Zé

Oferta da semana

Arroz 5 kg

de R$ 30,00

por R$ 27,00

Adicionar

Sem informar o desconto, vale o **padrão**: 10%.

Menos argumentos, menos erro.

```
function comDesconto(preco,
                     pct = 10) {
  return preco * (1 - pct / 100)
}
comDesconto(30)      // 27
comDesconto(30, 50) // 15
```

> erro_comum

console.log não é return

✗ Evite

```
function dobro(n) {
  console.log(n * 2)
}
const x = dobro(4)
// x: undefined
```

Mostrou na tela, mas não **devolveu**.

✓ Prefira

```
function dobro(n) {
  return n * 2
}
const x = dobro(4)
// x: 8
```

`return` entrega o valor a quem chamou.

04

Arrow functions

A forma curta, que você vai ver em todo lugar



Arrow function é um jeito **curto** de escrever funções.

Mesma função, três formas. A 3ª é a mais usada.

```
// 1. function clássica
function dobro(n) {
  return n * 2
}
// 2. arrow com bloco
const dobro = (n) => {
  return n * 2
}
// 3. arrow enxuta
const dobro = n => n * 2
```




Regras da arrow

`n =>`

Um parâmetro
parênteses são opcionais

`() =>`

Nenhum ou vários
parênteses obrigatórios

`=> x`

Uma expressão só
o return fica implícito

`=> ({})`

Devolver objeto
envolva o objeto em parênteses



> checkpoint

O que retorna?

```
const soma = (a, b) => a + b
soma(2, '3')
```

A `5`

B `'23'`

C `'5'`

D `NaN`

> resposta

O que retorna?

```
const soma = (a, b) => a + b
soma(2, '3')
```

A `5`

B `'23'`

C `'5'`

D `NaN`

05

Escopo

Onde cada variável existe

`if` `for` `fn` `=>` `{ }` `2x`

> analogia

Escopo: quem enxerga quem

No código

```
const cidade = 'Palmares'
function casa() {
  const quarto = 'cama'
  console.log(cidade) // ok
}
console.log(quarto)
// ReferenceError
```

≈

No dia a dia



De dentro de casa você **vê a rua**.

Da rua, ninguém vê o seu quarto.

let respeita o bloco

✗ Evite

```
if (true) {
  var vazou = 'oi'
}
console.log(vazou)
// 'oi' (escapou!)
```

var ignora as chaves do bloco.

✓ Prefira

```
if (true) {
  let preso = 'oi'
}
console.log(preso)
// ReferenceError
```

let e **const** ficam **dentro** do bloco.



> checkpoint

O que aparece no console?

```
let x = 1
if (true) { let x = 2 }
console.log(x)
```

A 1

B 2

C undefined

D ReferenceError

> resposta

O que aparece no console?

```
let x = 1
if (true) { let x = 2 }
console.log(x)
```

A 1

B 2

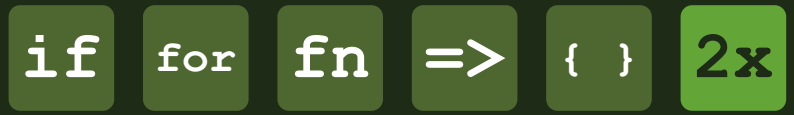
C undefined

D ReferenceError

06

Problemas em duplas

Três desafios com entrega no Classroom



Programação em par

P — **Piloto**
digita e fala o que está fazendo

N — **Navegador**
revisa e pensa no próximo passo

⇄ — **Troca a cada 10 min**
os dois pilotam, os dois pensam

? — **Travou?**
pergunte à dupla, depois ao professor



> desafio

Par ou ímpar?

⏱ 15 min

1. Crie **`classificar(n)`** com **`function`**.
2. Use `%` para descobrir se é par.
3. Devolva **`'par'`** ou **`'ímpar'`** com ternário.
4. Teste com 7, -4 e 0.

Bônus: diga também se é positivo ou negativo.

```
function classificar(n) {
  // sua vez!
}

classificar(7)    // 'ímpar'
classificar(-4)   // 'par'
classificar(0)    // 'par'
```




> desafio

Tabuada com for

⏱ 15 min

1. Crie a arrow **`tabuada(n)`**.
2. Use **`for`** de 1 a 10.
3. Mostre cada linha com template literal.
4. Chame para o 7 e para o 9.

Bônus: aceite o limite como parâmetro padrão.

```
const tabuada = (n) => {
  // sua vez!
}

tabuada(7)
// 7 x 1 = 7
// 7 x 2 = 14
// ...
// 7 x 10 = 70
```

> desafio

Caixa eletrônico

⏱ 25 min

1. Crie **`sacar(valor)`**.
2. Notas disponíveis: 100, 50, 20 e 10.
3. Entregue o menor número de notas.
4. Valor que não fecha? Avise com **`if`**.

```
const notas = [100, 50, 20, 10]

sacar(380)
// 3 de 100, 1 de 50,
// 1 de 20, 1 de 10

sacar(15)
// 'Valor inválido'
```

Entrega: link do código no Classroom.



> resumo

Leve para casa

if

if, **else** e ternário decidem o caminho.

for

for, **while** e **for...of** repetem sem copiar.

fn

Funções com **return** reaproveitam a lógica.



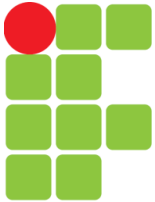
> próxima_aula

Arrays e métodos funcionais

14/10 · sala de aula invertida

Antes da aula: leia MDN · Array (map, filter)
Termine os desafios no Classroom

Prof. Thiago Araújo · IFPE Campus Palmares · ADS 3º período



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO